Cluster: AIML

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PULCHOWK CAMPUS

Natural Language to OKS Query Translation Module

SUBMITTED BY

Asmit Khanal (PUL08BCT017)

Chandan Kumar Shah (PUL080BCT023)

Aditya Kumar Shah (PUL080BCT011)

Rhythm Adhikari (PUL080BCT065)

A

BE MINOR PROJECT FINAL REPORT

SUBMITTED TO THE

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF ENGINEERING IN

COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

LALITPUR, NEPAL

AUGUST 26, 2026

DECLARATION OF AUTHORSHIP

This is to certify that the minor project entitled "**Natural Language to OKS Query Translation Module**" submitted for the degree of Bachelor of Engineering in Computer Engineering comprises our original work and no part of this work has been used for the award of another course or degree. We declare that this work is our own. If any part of the work is not our own, we have indicated it by acknowledging the source of that part or those part of the work with proper citations.

Name of the Student	**Signature**
Asmit Khanal	_____
Chandan Kumar Shah	_____
Aditya Kumar Shah	_____
Rhythm Adhikari	_____

Date: August 26, 2026

APPROVAL

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a minor project entitled "**Natural Language to OKS Query Translation Module**" submitted by Asmit Khanal (PUL08BCT017), Chandan Kumar Shah (PUL080BCT023), Aditya Kumar Shah (PUL080BCT011), and Rhythm Adhikari (PUL080BCT065) in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Supervisor, Dr. Basant Joshi
Assistant Professor,
Department of Electronics & Computer Engineering
Pulchowk Campus, IOE, TU

Internal Examiner,
Department of Electronics & Computer Engineering
Pulchowk Campus, IOE, TU

Head of Department, Prof. Dr. Ram Krishna Maharjan
Department of Electronics & Computer Engineering,
Pulchowk Campus, IOE, TU

Date: August 26, 2026

COPYRIGHT

ABSTRACT

The ATLAS Trigger and Data Acquisition (TDAQ) system uses the Object Kernel Support (OKS) framework to store a large, versioned graph of configuration objects. Although OKS provides precise and efficient access for expert users, its proprietary OksQuery language is difficult to formulate manually and is not known reliably by general-purpose language models. This project develops a read-only Natural Language to OksQuery translation module exposed through a Model Context Protocol (MCP) server.

The implementation uses version-scoped schema retrieval, few-shot examples, structured intermediate representation, deterministic schema validation, a bounded repair loop, and read-only execution through the TDAQ Python interface or native `oks_dump`. The MCP server runs on CERN LXPLUS and is reached from a workstation through an SSH local port forward. The server returns the generated query, target class, result count, filtered objects, and context diagnostics while leaving conversation memory to the calling agent. Runtime demonstrations show successful tool discovery, a valid empty result, and a query returning 16 `BaseApplication` identifiers. These demonstrations validate the integration path; a statistically powered accuracy benchmark remains future work.

Keywords: OKS, OksQuery, ATLAS TDAQ, natural-language query, retrieval-augmented generation, Model Context Protocol, schema validation.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor and the Department of Electronics and Computer Engineering, Pulchowk Campus, for their guidance and support throughout this project. We also acknowledge the ATLAS TDAQ and CERN computing environments that made it possible to study OKS configuration data and verify the implementation on LXPLUS. Finally, we thank our project members, friends, and families for their encouragement during the development and documentation of this work.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

AI	Artificial Intelligence
AIML	Artificial Intelligence and Machine Learning
API	Application Programming Interface
AST	Abstract Syntax Tree
ATLAS	A Toroidal LHC ApparatuS
BM25	Best Matching 25
CERN	European Organization for Nuclear Research
CLI	Command-Line Interface
CVMFS	CernVM File System
DAQ	Data Acquisition
DFD	Data-Flow Diagram
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
IR	Intermediate Representation
JSON	JavaScript Object Notation
LHC	Large Hadron Collider
LLM	Large Language Model
MCP	Model Context Protocol
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OKS	Object Kernel Support
RAG	Retrieval-Augmented Generation
SDK	Software Development Kit
SQL	Structured Query Language
SSH	Secure Shell
TDAQ	Trigger and Data Acquisition
TLS	Transport Layer Security
XML	Extensible Markup Language

CHAPTER 1

INTRODUCTION

1.1 Background

The ATLAS experiment at CERN's Large Hadron Collider (LHC) relies on a highly complex data acquisition (DAQ) system to manage thousands of hardware and software components in real time. The configuration of this system, spanning detector electronics, readout chains, and computing farms, is managed by the Object Kernel Support (OKS) framework, a persistent in-memory object database built for the ATLAS Trigger and Data Acquisition (TDAQ) system [1]. OKS provides a schema-driven, object-oriented model that represents complex topologies as interconnected object graphs [1].

To meet the operational demands of successive LHC runs, ATLAS configuration management evolved from monolithic databases [2] to a Git-based service for LHC Run 3 [4]. This modern architecture provides distributed version control and full provenance tracking, ensuring every configuration state is a Git commit and the entire detector history is queryable [4].

Despite this mature infrastructure, querying the OKS database remains a specialist task. The proprietary OksQuery language uses a strict, S-expression-like syntax (e.g., `(all ("InitTimeout" "2" >))`) that demands detailed knowledge of class hierarchies, relationship paths, and operator semantics. Consequently, manual query formulation by shifters is error-prone, time-consuming, and inaccessible to non-specialists.

The emergence of Large Language Models (LLMs) and the Model Context Protocol (MCP) offers a new paradigm for AI-driven operations, enabling agents to interact with external databases through a standardized interface. An MCP server for the ATLAS DAQ is currently under development to allow LLMs to search the OKS database. However, a critical challenge arises: the full configuration, comprising hundreds of schema classes and thousands of objects, far exceeds any LLM's context window. Thus, the MCP server must filter data on the backend, requiring the LLM to generate syntactically perfect OksQuery strings without seeing the complete schema.

This project addresses this challenge by developing a Natural Language to OksQuery translation module. Architected as a Retrieval-Augmented Generation (RAG) system, it

utilizes a “schema-RAG” approach where the retrieval corpus is the live, version-scoped OKS schema itself. This ensures the LLM receives only the relevant schema slice (1–3 classes) rather than the full schema, which can exceed 500 classes depending on the TDAQ release and partition, satisfying both accuracy and context-window constraints. The system is deployed securely within the CERN environment via SSH-based hosting with port tunneling, exposing the MCP server to external LLM agents without requiring them to reside inside the CERN network perimeter.

1.2 Problem Statement

The Object Kernel Support (OKS) framework is a persistent in-memory object database that provides a schema-driven, object-oriented configuration model. By supporting classes, attributes, inheritance, and composite objects, it enables the precise representation of complex hardware and software topologies. However, interacting with OKS remains a highly specialized task. Users must formulate queries using the proprietary OksQuery language or low-level C++ and Python APIs, which demands detailed, manual knowledge of class hierarchies, object states, relationship paths, and strict operator semantics.

While general-purpose Large Language Models (LLMs) present an opportunity to automate this process, they are fundamentally incapable of generating accurate OksQuery statements out-of-the-box. This failure is driven by a complete lack of parametric context regarding the specific classes, attributes, and objects within the OKS schema. Furthermore, the configuration is strictly version-specific, with schema definitions evolving across different software releases and Git commits. Because LLMs lack access to this dynamic, version-scoped context, they frequently hallucinate non-existent classes or invalid relationships, rendering their raw query accuracy insufficient for operational deployment.

To address this, we built a query translation module hosted as an MCP server via SSH port tunneling, using a version-scoped schema-RAG pipeline that injects only the relevant OKS schema slice into the LLM prompt. A deterministic validate/repair loop then checks every generated OksQuery against the authoritative schema before execution, ensuring reliable accuracy without the full configuration entering the context window.

1.3 Objectives

1.3.1 Main Objective

The main objective of this project is to develop an MCP server capable of translating natural-language questions into validated OksQuery statements through a query translation module with an integrated validate/repair loop, deployable as a tool for external LLM agents.

1.3.2 Specific Objectives

The specific objectives of this project are:

(i) To develop a version-scoped pipeline that translates natural language into OksQuery strings by retrieving targeted OKS schema slices and enforcing syntactic correctness through a deterministic validate/repair loop.

(ii) To deploy the translation module as a Model Context Protocol (MCP) server within the CERN compute infrastructure, utilizing SSH port tunneling to securely expose the live TDAQ configuration to external LLM agents.

(iii) To evaluate translation accuracy by executing generated queries against the native C++ backend and analyzing performance metrics such as valid-syntax rate, result-set equivalence, and execution latency across stratified query difficulties.

1.4 Rationale

The ATLAS Data Acquisition (DAQ) system relies on the Object Kernel Support (OKS) framework, a complex, version-controlled object database. Querying this configuration during time-critical operations requires specialized knowledge of the proprietary OksQuery language, creating a significant bottleneck for control room operators.

While Large Language Models (LLMs) could automate this querying process through natural language, raw models fail because they hallucinate syntax for undocumented, proprietary languages and cannot ingest massive, constantly evolving schemas within their context limits.

This project addresses these fundamental limitations by developing a retrieval-augmented translation module deployed as a Model Context Protocol (MCP) server. By utilizing a version-scoped Schema-RAG pipeline, the system feeds the LLM only the

exact schema slice it needs. Furthermore, by forcing the LLM to output an Abstract Syntax Tree (AST) that undergoes a strict validate-and-repair loop against the authoritative, version-bound OKS schema, the system eliminates syntax hallucinations before execution. Ultimately, this establishes a robust and verifiable paradigm for safely deploying generative AI within high-stakes scientific infrastructure.

1.5 Scope

The system is scoped to translating natural-language questions about the ATLAS DAQ configuration into validated OksQuery statements for read-only retrieval, processing one query at a time within a single, explicitly resolved configuration version (TDAQ release, Git revision, or run number), with schema retrieval performed dynamically per request using a version-scoped lexical pipeline (exact match, alias lookup, BM25) keyed by schema fingerprint, and the retrieval index regenerated from the OKS schema whenever the configuration changes. The module generates an intermediate AST representation rather than raw OksQuery strings, validated deterministically against the version-bound OKS schema before compilation, and is deployed as an MCP server hosted within CERN compute infrastructure and exposed to external LLM agents through SSH port tunneling, with conversational persistence and session state managed by the consuming LLM agent rather than the translation module. The system does not write, modify, or delete configuration data, does not provide operational recommendations or interpret query results beyond factual reporting, does not access live running partition memory or process telemetry data, does not handle user authentication (security relies on the CERN SSH tunnel and institutional network perimeter), and does not support non-English natural-language input or cross-version comparison queries.

1.6 Limitations

The present system is a read-only research prototype. Its limitations are as follows:

(i) Query generation is evaluated through selected demonstrations and integration tests, not through a large, independently labelled benchmark with confidence intervals.

(ii) The service depends on the availability of a compatible TDAQ release, OKS data file, Python bindings, native tools, and an OpenAI-compatible LLM endpoint.

Results can change when the selected configuration version changes.

(iii) The server is stateless and expects the calling agent to resolve follow-up questions into complete requests. It does not maintain user sessions or conversation history.

(iv) The current deployment uses a loopback-bound server and SSH tunnelling for private development access. Authentication, authorization, persistent supervision, and public-service hardening are outside the prototype scope.

(v) The implementation supports English natural-language requests and read-only configuration queries; it does not access live process telemetry, modify configuration, or compare multiple versions in one request.

1.7 Outline of the Report

Chapter 1 introduces the ATLAS TDAQ configuration problem, project objectives, scope, and limitations. Chapter 2 reviews OKS, ATLAS configuration management, retrieval-augmented generation, few-shot prompting, and related query translation research. Chapter 3 presents the methodology, architecture, modules, data flow, testing strategy, and deployment setup. Chapter 4 maps that design to the implemented Python package and MCP service, including the runtime interface and deployment challenges. Chapter 5 reports the verified integration results and discusses what the demonstrations do and do not prove. Chapter 6 concludes the report and recommends future evaluation and production hardening work.

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Related Theory

The proposed project develops a natural-language interface for querying the Object Kernel Support (OKS) configuration used by the ATLAS Trigger and Data Acquisition (TDAQ) system. OKS is not a conventional relational database. Jones *et al.* describe it as a persistent in-memory object manager designed for fast access to structured information in configuration and real-time control applications [1]. Its object model supports classes, object identifiers, associations, inheritance, polymorphism, integrity constraints, and schema evolution [1]. Therefore, an OKS query depends on relationships among objects and on the schema version in which those objects are defined.

The ATLAS literature demonstrates the scale of this problem. Almeida *et al.* describe an online configuration service that had to represent information from several DAQ, trigger, and detector groups, preserve historical configurations, and serve many processes during online operation [2]. The configuration contains interconnected classes and objects rather than isolated records. A user's question may consequently require a target class, inherited attributes, related classes, object identifiers, and valid operators at the same time. A language model that generates a query without the relevant schema may produce a syntactically plausible but semantically invalid expression.

Configuration correctness is also a governance concern. Soloviev reports that the ATLAS version-control service was introduced to control access, preserve modification history, and detect XML syntax errors and inconsistent references [3]. The later Git-based service for LHC Run 3 continues to emphasise consistency, performance, access control, traceability, and archiving while managing more than one thousand interconnected XML files [4]. These findings support two requirements for the proposed translator: queries must use an explicitly selected release or revision, and generated queries must pass deterministic validation before execution.

2.2 Retrieval-Augmented Generation and Few-Shot Prompting

Large language models can translate natural-language instructions, but their internal knowledge cannot reliably contain a private and changing OKS schema. Retrieval-

Augmented Generation (RAG) addresses this limitation by combining a language model with external knowledge retrieved at query time. Lewis *et al.* define RAG as a combination of parametric model memory and non-parametric external memory, and show that retrieval can improve factual specificity in knowledge-intensive tasks [5]. For this project, the external memory is the selected OKS schema and configuration metadata, including class definitions, inherited members, relationships, object identifiers, operators, and verified query examples.

RAG does not guarantee correctness by itself. If the retriever selects the wrong class, omits a relationship, or mixes information from different releases, the model may generate an invalid query. Retrieval must therefore be version-aware and evaluated separately from generation. The system should measure whether the relevant classes, attributes, and relationships were retrieved before measuring the final execution result.

Few-shot prompting is also relevant. Brown *et al.* show that large language models can perform many tasks from instructions and a small number of examples without task-specific parameter updates [6]. In this project, examples can demonstrate OksQuery syntax and structure. However, examples are guidance rather than authority: an example from an older TDAQ release may contain a class or attribute unavailable in the current release. Demonstrations should therefore be filtered using the same version metadata as the retrieved schema.

2.3 Natural-Language Query Translation and Validation

Natural-language-to-SQL research provides a useful comparison, although OKS is object-oriented and is not equivalent to SQL. Katsogiannis-Meimarakis and Koutrika identify schema linking—the association of terms in a question with database elements—as a central stage in neural text-to-SQL systems [7]. They also highlight query representation, generation, and execution-based evaluation [7]. These principles transfer to OKS, where schema linking must identify classes, attributes, relationship paths, values, and object identifiers.

Direct generation of a raw OksQuery string is risky because syntax validity is only one part of correctness. A query may be syntactically valid but use a nonexistent attribute, an incompatible operator, an incorrect relationship target, or an object absent from the selected configuration. The proposed system should therefore generate a typed inter-

mediate representation (IR), such as JSON or an abstract syntax tree, before serializing it into OksQuery. The IR can represent the target class, predicates, relationship traversal, values, version, and result limit. A deterministic validator can then check class and attribute existence, relationship typing, operator/value compatibility, and version consistency. Invalid output can be repaired a limited number of times or rejected safely.

Evaluation should focus on execution accuracy rather than exact text matching. Equivalent queries may have different textual forms. The main measure should be whether the validated query runs against the intended configuration version and returns the expected objects. Supporting measures should include schema-linking accuracy, first-pass validation rate, repair success rate, execution latency, and safe-rejection accuracy.

2.4 Research Gap and Project Relevance

The reviewed literature provides complementary foundations but does not directly evaluate a natural-language interface for versioned ATLAS OKS configuration queries. The OKS and ATLAS studies establish the domain requirements of interconnected objects, schema evolution, historical access, and consistency control [1–4]. RAG and few-shot learning provide methods for grounding a language model and adapting it to a specialised query format [5, 6]. Text-to-SQL research contributes schema-linking and execution-based evaluation principles [7].

The reviewed sources did not identify one evaluated pipeline combining version-scoped OKS schema retrieval, few-shot translation, typed query generation, deterministic schema/type validation, bounded repair, and execution against the matching configuration. This project addresses that integration gap as a read-only research prototype. Its contribution is not a new language model; it is an auditable translation and execution pipeline intended to make LLM-assisted OKS querying more reliable. The system should return the generated query with the selected release or revision, validation outcome, and result provenance. If the required context is missing or inconsistent, it should reject the request rather than execute an unverified query.

CHAPTER 3

METHODOLOGY

3.1 Project approach

The project was developed through successive capability increments and repeated verification against the OKS runtime. The work began with a translation prototype and was refined when testing exposed limitations in query generation, schema awareness, version handling, and execution. Each refinement preserved the working components while adding a more reliable capability. The final implementation therefore combines an iterative development process with an incremental growth of functionality, rather than treating the system as a single implementation completed at the end of the project.

The principal stages of the approach were as follows:

I. **Domain and runtime analysis.** The OKS object model, schema files, OksQuery grammar, Python `config` binding, and native `oks_dump` utility were examined first. This established the supported classes, attributes, relationships, operators, query constraints, and runtime requirements.

II. **Baseline translation prototype.** A basic natural-language to OksQuery path was implemented to test the feasibility of using an OpenAI-compatible language model for query generation. Initial outputs were inspected and executed against the native runtime so that syntactic and semantic failure cases could be identified.

III. **Version-scoped context construction.** The approach was then extended to resolve current and historical configuration versions. For each request, the context builder creates an immutable `OksContext` with version metadata, schema and data locations, and a schema fingerprint. The retrieval index is consequently aligned with the configuration used for execution.

IV. **Structured translation and validation.** Instead of asking the language model to produce executable query text directly, the system requests a JSON intermediate representation. The representation is normalised, checked structurally and semantically against the active schema, repaired within a bounded retry loop when required, and compiled deterministically to OksQuery.

V. **Execution, service integration, and verification.** Validated queries are ex-

ecuted through the Python `config` interface or the `oks_dump` fallback. The pipeline was subsequently exposed through the stateless MCP service, which provides query, translation-only, and environment-probe tools. Unit, integration, and MCP contract tests were used to verify the individual modules and their combined behaviour.

3.2 System Architecture

The system follows a service-oriented pipeline deployed across the user environment and the CERN TDAQ environment. A user submits a natural-language question through a coding agent or another MCP-compatible client. The client connects to a local MCP endpoint, which is securely forwarded to the MCP server running on CERN lxplus through an SSH local port forward. This keeps the remote service private and avoids exposing a public application port.

Within the CERN environment, the Streamable HTTP MCP server exposes the `oks_query`, `oks_translate`, and `oks_environment_probe` tools. `OksQueryService` validates inputs, applies request and result limits, serialises the request, and returns structured responses. The `OksPipeline` resolves the requested configuration, retrieves the relevant version-scoped schema context, and coordinates translation and execution.

The translation and validation stage communicates with an OpenAI-compatible LLM API over HTTPS. The generated representation is checked and repaired when necessary before compilation. The executor then accesses the approved OKS data through the TDAQ Python `config` binding or the native `oks_dump` fallback. Only filtered structured results and diagnostic metadata are returned to the MCP client; conversational memory remains with the calling agent.

The deployment and request flow are summarised in Figure 3.1.

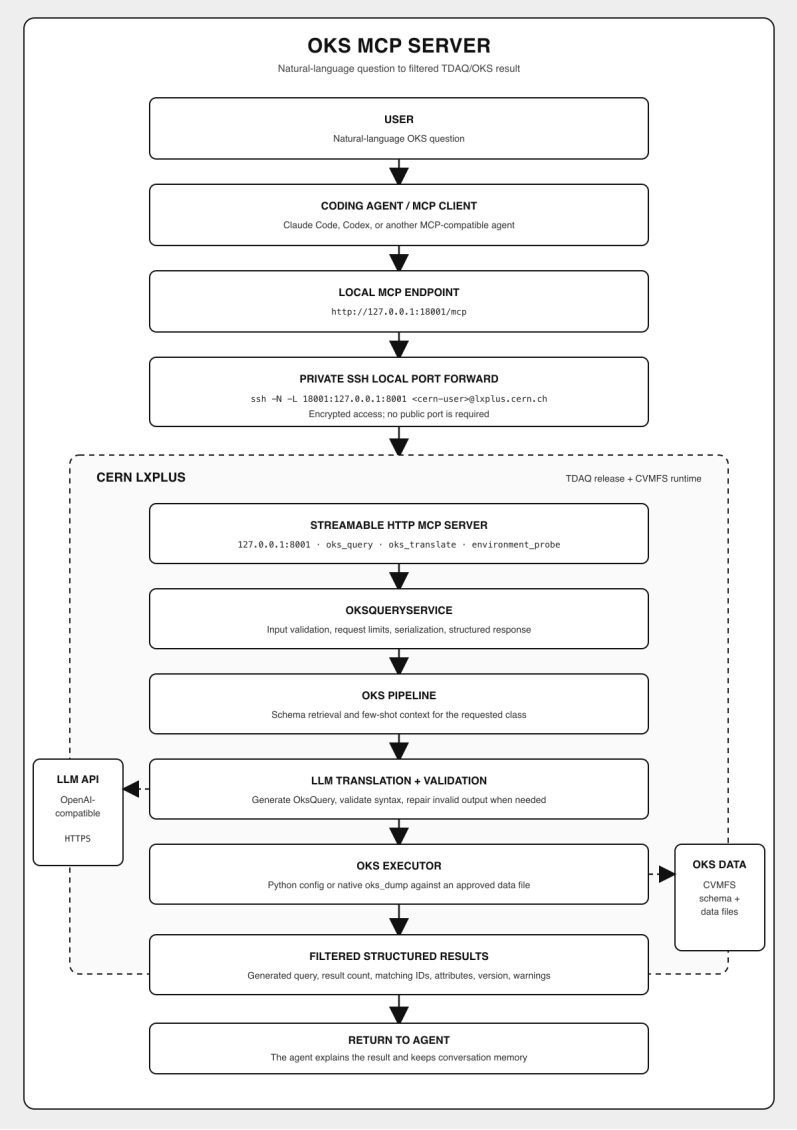

Figure 3.1: Deployment and processing architecture of the OKS MCP server.

3.3 System Design and Modules

The system is a layered, read-only translation and execution pipeline. It separates request handling, version resolution, schema retrieval, model interaction, deterministic validation, execution, and MCP service access so that each responsibility can be tested independently.

I. **Request and intent module.** CLI, programmatic, and MCP entry points validate the question, version selector, and service limits. Intent classification separates query, help, and out-of-scope requests.

II. **Version and context module.** The resolver interprets current or historical tags, hashes, dates, and run identifiers. The context builder creates the version-bound `OksContext`, with a request lock protecting historical runtime selection.

III. **Schema retrieval module.** The provider reads classes, attributes, inheritance, and relationships from the active schema. Exact and lexical matching, synonym hints, and relationship expansion build a compact, version-fingerprinted schema context.

IV. **Translation and validation module.** The prompt builder combines the question, schema context, rules, and examples. The translator creates a structured representation; `oks_ast` validates it and the compiler deterministically produces OksQuery text. Invalid output enters a bounded repair loop.

V. **Execution and interpretation module.** The executor runs only validated queries through the TDAQ Python `config` binding or `oks_dump`. Results include query, version, target class, count, status, and warnings. CLI output may be interpreted in the application, while MCP returns structured data to the calling agent.

VI. **MCP service module.** `OksQueryService` provides a stateless boundary with input and version checks, result limits, serialisation, and controlled errors. FastMCP exposes `oks_query`, `oks_translate`, and `oks_environment_probe`; configuration remains in OKS/TDAQ.

3.3.1 Use-Case Diagram

The use-case view identifies the developer or team member and coding agent as the main actors. The LLM API supports translation and repair, while the server handles MCP

access, versioned queries, validation, environment probing, and read-only execution. Conversation memory and follow-up resolution remain with the host agent; the server exposes only approved data and filtered results.

Use-Case Descriptions

UC-01: Query OKS Data **Actor:** User, programmatic client, or MCP client.
Precondition: The service is configured and the question and selected configuration are valid.
Postcondition: Structured results, status, and warnings are returned; the OKS configuration is unchanged.
Flow: Validate the request, resolve context, retrieve schema, generate and validate the representation, compile OksQuery, execute it, and return the normalised result. Unresolved versions, runtime failures, or exhausted repair attempts stop execution with a controlled error.

UC-02: Translate Without Execution **Actor:** User, developer, or MCP client.
Precondition: The question, optional selector, schema context, and LLM endpoint are available.
Postcondition: A validated representation and compiled OksQuery are returned without contacting the OKS runtime.
Flow: Resolve context, retrieve schema, generate, validate, and if necessary repair the representation within the retry limit. Invalid input, missing schema data, or failed repair returns a controlled error.

UC-03: Probe OKS Environment **Actor:** User, operator, or MCP client.
Precondition: Runtime paths and settings are loaded.
Postcondition: A non-secret readiness report covers dependencies, paths, schema availability, and classes; no query or mutation occurs.
Flow: Check the Python `config` path and/or `oks_dump`; report missing dependencies as diagnostics.

Included use cases **UC-04: Resolve Context** interprets a current or historical selector and creates a version-bound `OksContext`. **UC-05: Validate and Compile** normalises

the JSON representation, checks it against the active schema, and compiles it only after validation; invalid output is repaired within a bound. **UC-06: Execute Query** runs the validated expression through the Python `config` binding or `oks_dump` and returns a read-only `ExecutionResult`.

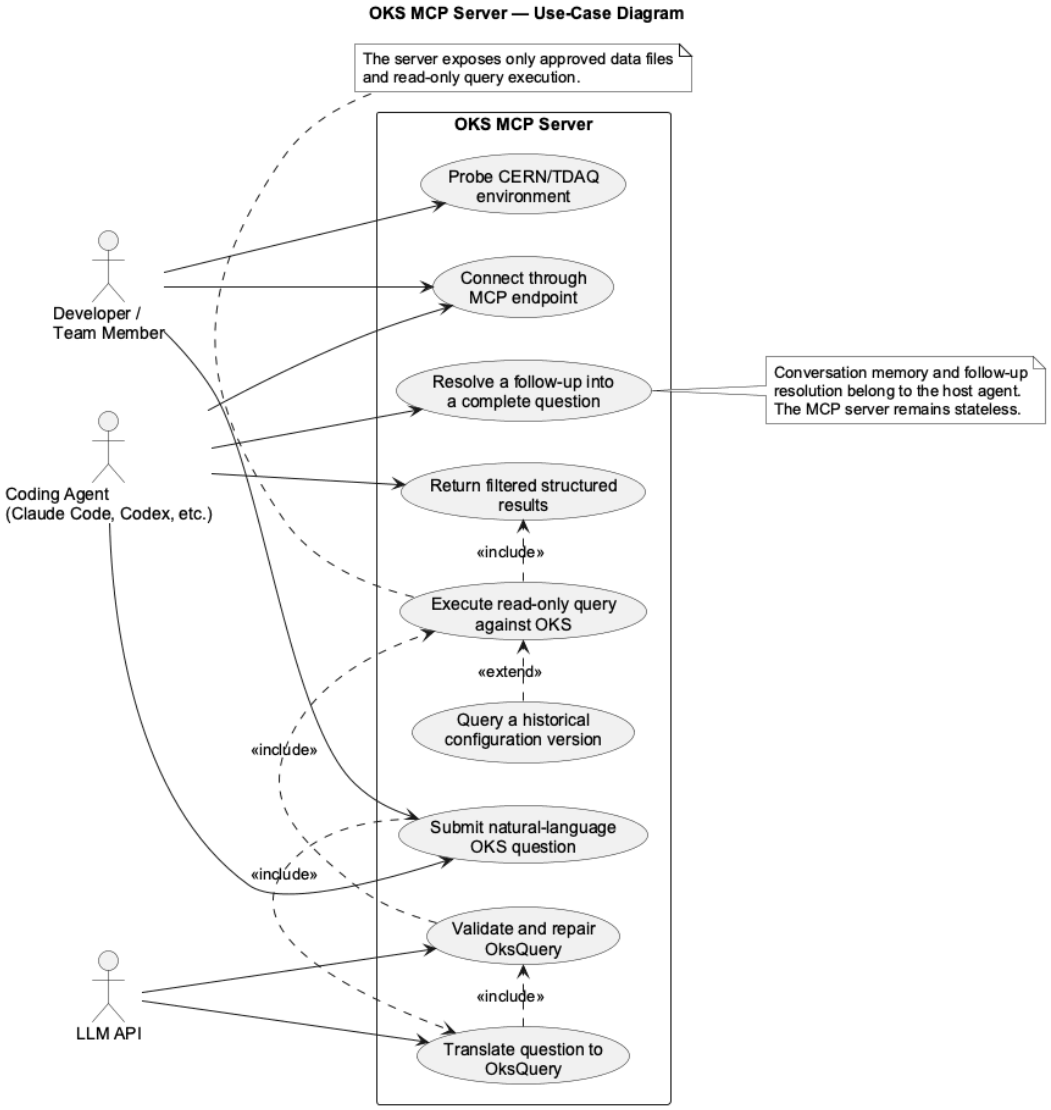

Figure 3.2: Use-case diagram of the OKS MCP Server.

3.3.2 Context Diagram (DFD Level 0)

The Level-0 DFD models the OKS MCP Server as one process interacting with the developer or coding agent, the LLM API, and the CERN TDAQ/OKS configuration. It receives a question and version, retrieves schema context, validates the candidate query, executes only an approved read-only request, and returns filtered structured results. Levels 1 and 2 are detailed in Data Handling.

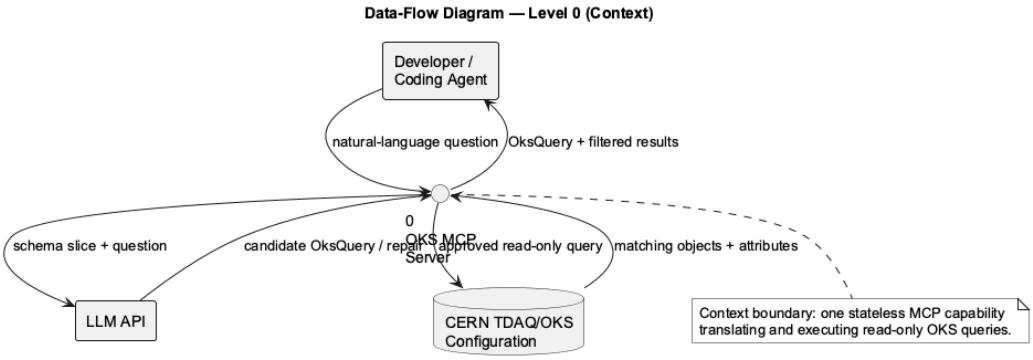

Figure 3.3: Level-0 context data-flow diagram of the OKS MCP Server.

3.3.3 Class Diagram

The class view shows the MCP adapter, service facade, pipeline, and supporting translation, retrieval, validation, and execution components. `OksContext` carries version and schema metadata, while `ExecutionResult` records filtered objects, status, and version. The separation supports independent testing of request handling, model generation, schema checks, and runtime access.

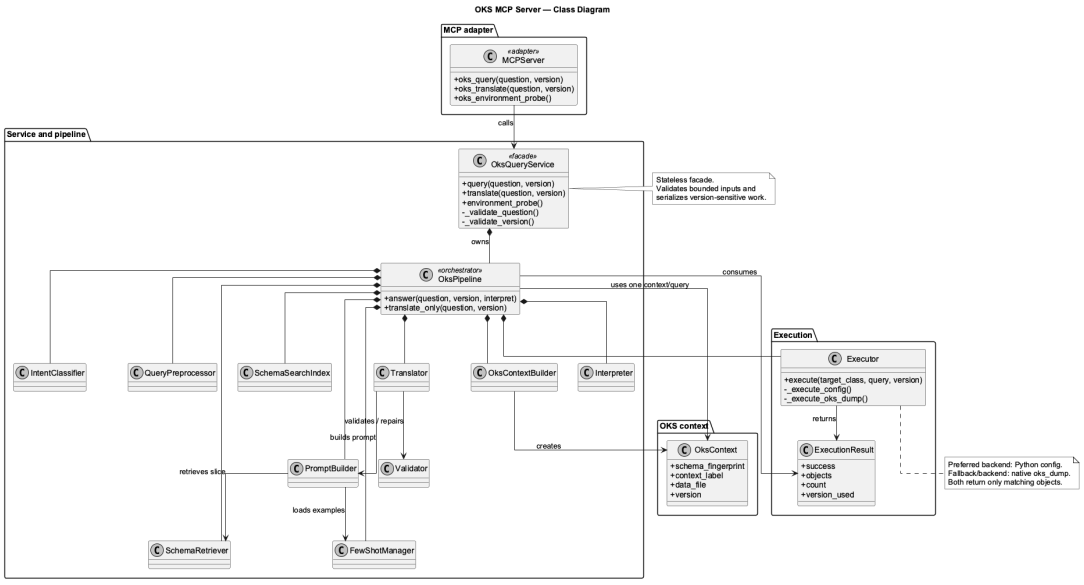

Figure 3.4: Principal classes and collaborations in the merged implementation.

3.3.4 Sequence Diagram

The sequence view shows the deployed path from the user and coding agent, through the local SSH port forward, to the MCP service and pipeline on CERN. The pipeline obtains schema context, asks the LLM for a candidate, validates or repairs it, and exe-

cutes only the approved read-only query against TDAQ/OKS. Filtered structured results return through the tunnel for agent interpretation.

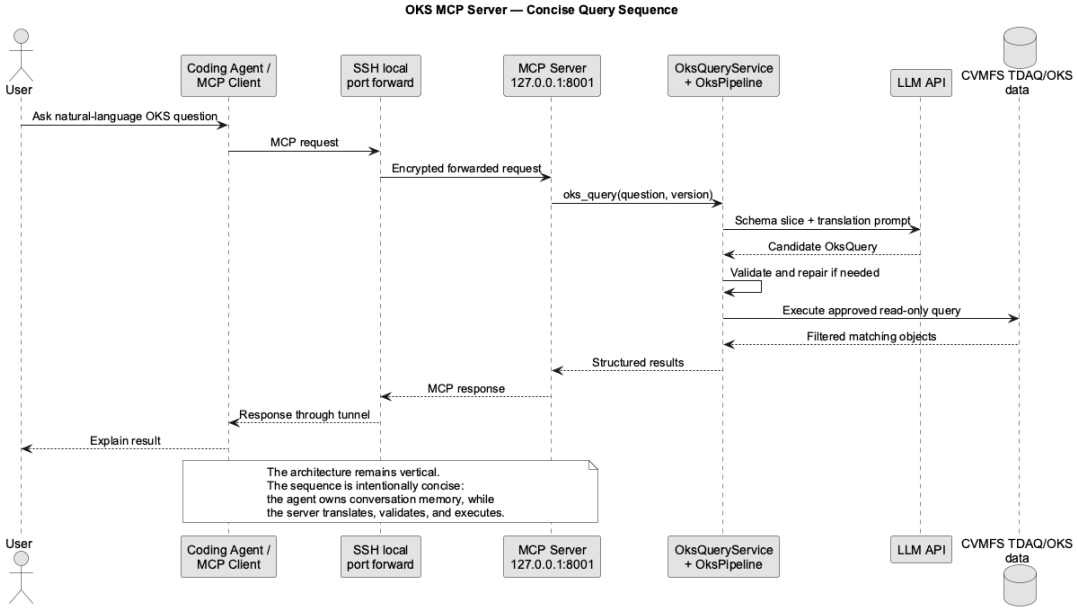

Figure 3.5: Concise sequence of operations for an MCP query request.

3.4 Technology Stack

The system uses the following technologies to support OKS integration, structured translation, and MCP service delivery:

I. **Python.** Python 3.10 or later is used for the pipeline, service layer, schema processing, validation, and testing.

II. **OKS/TDAQ integration.** The TDAQ Python `config` binding provides the primary execution interface, with `oks_dump` as a fallback and diagnostic tool.

III. **LLM and structured validation.** The `openai` package connects to an OpenAI-compatible endpoint. JSON, Pydantic, and the project `oks_ast` modules support normalisation, semantic validation, repair, and deterministic compilation.

IV. **Schema retrieval.** Versioned OKS schemas and Git/runtime metadata provide context. Lexical retrieval and `rank_bm25` identify relevant classes, attributes, and relationships.

V. **MCP and deployment.** The Python MCP SDK with FastMCP exposes the query, translation, and environment-probe tools through `stdio` or Streamable HTTP. Environment variables, `python-dotenv`, SSH, and `tmux` support configuration and deployment.

VI. **Testing.** `pytest` is used for unit, integration, service, pipeline, and MCP contract tests.

3.5 Development Methodology

The project used iterative prototyping with incremental refinement rather than a strict Agile or Waterfall process. Requirements were clarified through implementation and testing, and the increments were not separately deployed. A prototype was tested against native OKS tools; identified limitations then guided version-aware retrieval, validation and repair, execution, and MCP integration.

$$\textit{Prototype} \rightarrow \textit{Test} \rightarrow \textit{Refine} \rightarrow \textit{Integrate MCP} \rightarrow \textit{Validate}$$

3.6 Data Handling

The system handles natural-language questions and approved, version-scoped OKS data without introducing an application database.

3.6.1 Input and Storage

The input is a complete question with an optional current or historical selector. The service validates the question, selector, input length, and result limit; the operator, not the MCP client, chooses the approved data file. TDAQ/OKS schemas and data files remain authoritative. A request-specific schema index, examples, context metadata, and fingerprint are held in memory, while conversation history and configuration objects are not persisted.

3.6.2 Processing and Context Management

The question is classified and used to retrieve a compact, fingerprint-scoped schema slice through exact and lexical matching. The LLM receives only the question and relevant context, not the complete data file. Its JSON query IR is normalised, validated, and repaired within a bounded limit; the deterministic compiler then produces OksQuery. The executor runs the approved query through the Python `config` binding or `oks_dump` and returns bounded, read-only structured results.

3.6.3 Level-1 Data Flow

Figure 3.6 shows the main request flow: validate and classify the question, retrieve schema and examples, translate and validate the query, execute the approved expression against the selected data file, and serialise a bounded MCP response.

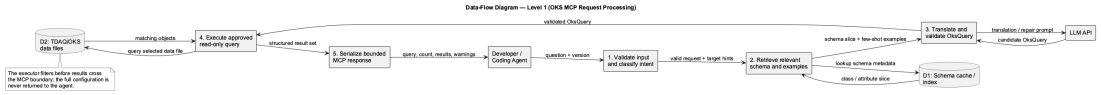

Figure 3.6: Level-1 data-flow diagram for OKS MCP request processing.

3.6.4 Level-2 Translation and Validation Flow

Figure 3.7 expands translation and validation. Target terms select a fingerprinted schema and prompt context; the LLM generates a candidate IR, which is checked for valid classes, attributes, operators, scope, and logic. Only the valid branch reaches compilation and execution; the invalid branch returns diagnostics to a bounded repair prompt.

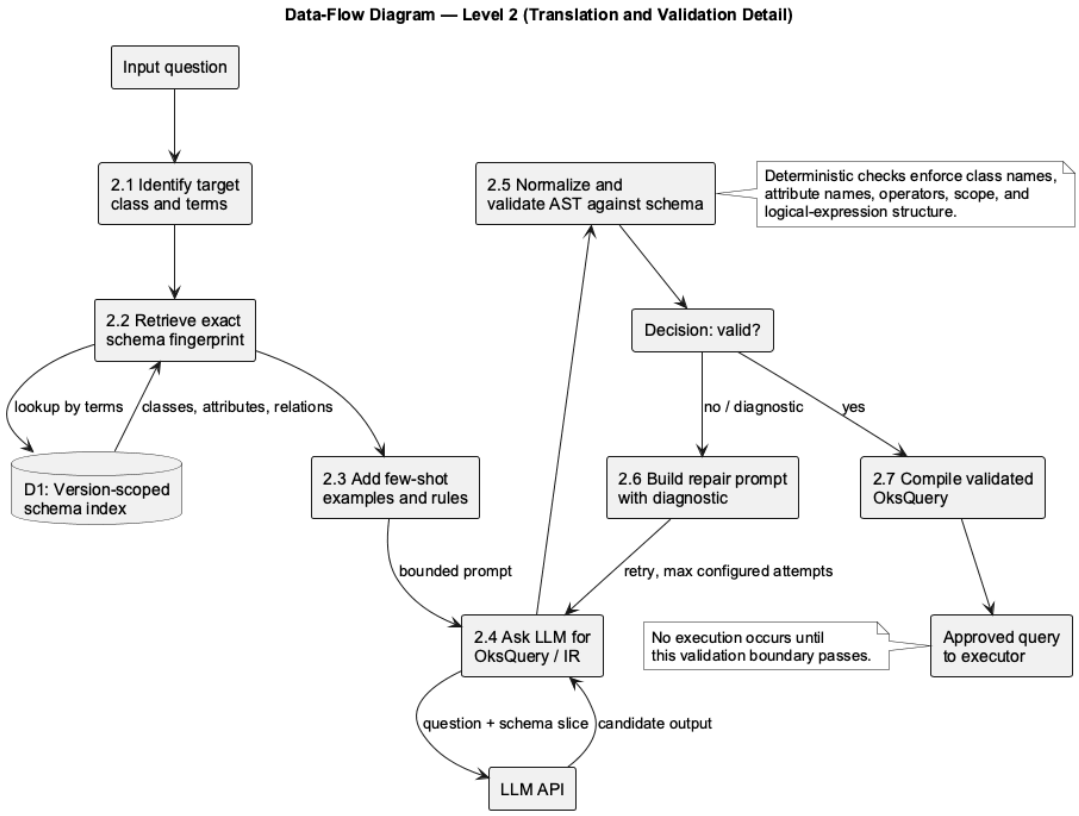

Figure 3.7: Level-2 data-flow diagram for translation and validation.

3.7 Testing and Validation

Testing covered components, the complete pipeline, the service boundary, and the runtime to prevent invalid queries from reaching OKS.

I. **Unit testing.** Intent, retrieval, version resolution, validation, compilation, and execution components are tested independently.

II. **Integration testing.** End-to-end tests cover context construction, translation, validation, execution, interpretation, and historical-version consistency.

III. **Service and MCP testing.** Input limits, version checks, result bounds, serialisation, errors, statelessness, and the three MCP tool contracts are verified.

IV. **Runtime and evaluation.** Queries are checked against native OKS tools and representative project cases. The MCP-agent workflow is verified for correctness, reproducibility, and safe read-only operation.

The `pytest` suite contains component, pipeline, service, architecture-invariant, and MCP-server tests.

3.8 Deployment Setup

The MCP server runs on CERN lxplus with the ATLAS TDAQ release and OKS files available through CVMFS. The workstation connects through an authenticated SSH tunnel to the loopback-bound MCP endpoint.

I. **Access and source.** Connect to lxplus through SSH, clone the final GitHub repository, and select the checkout containing the merged MCP implementation.

II. **Environment preparation.** Source one TDAQ release from CVMFS, create a Python 3.10-or-newer virtual environment, install requirements, and configure the LLM endpoint outside version control. Select the approved `OKS_DATA_FILE` and repository root.

III. **Server launch.** The launcher sources TDAQ and `deploy/start_mcp_tmux.sh` starts Streamable HTTP in detached `tmux` at `127.0.0.1:8001/mcp`; it is not publicly reachable.

IV. **SSH forwarding.** From the workstation, run `ssh -N -L 18001:127.0.0.1:8001`. The agent uses `http://127.0.0.1:18001/mcp`, while the CERN service remains private.

V. **Agent connection.** Register the forwarded endpoint in Claude Code or another MCP-compatible agent. It discovers `oks_query`, `oks_translate`, and `oks_environment_probe`; conversation memory remains with the agent.

VI. **Operations and security.** Check the listener and environment probe before use. Keep the service read-only and loopback-bound, store keys outside Git, and use an approved supervisor for persistent deployment.

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 Requirements Specification

The implementation requirements were derived from the need to allow an MCP-compatible agent to query the ATLAS TDAQ configuration without receiving the complete OKS configuration or obtaining direct access to the CERN environment. The system is intentionally limited to read-only retrieval of configuration objects.

4.1.1 Functional Requirements

I) Question acceptance: The system shall accept one complete natural-language question and an optional configuration-version selector.

II) Intent and schema retrieval: The system shall identify the query intent, construct a version-bound OKS context, and retrieve only the schema classes, attributes, and relations relevant to the question.

III) Query translation: The system shall use the retrieved schema slice and few-shot examples to translate the question into a structured intermediate representation and an OksQuery expression.

IV) Query validation and repair: The system shall normalize and validate the generated query against the selected OKS schema before execution. Invalid candidates shall be supplied with diagnostics to a bounded repair loop.

V) Read-only execution: The system shall execute only validated, read-only queries through the Python `config` interface or the native `oks_dump` utility.

VI) Structured response: The system shall return a structured response containing the generated OksQuery, target class, result count, matching object identifiers and selected attributes, version/context metadata, and warnings.

VII) MCP tool interface: The MCP interface shall expose `oks_query` for translation and execution, `oks_translate` for translation without execution, and `oks_environment_probe` for non-secret environment diagnostics.

4.1.2 Non-functional Requirements and Constraints

I) **Safety:** The MCP boundary shall not expose shell-command execution, arbitrary filesystem paths, raw unvalidated queries, or write operations.

II) **Context efficiency:** The full OKS configuration shall remain on the server. Only a small, query-relevant schema slice and filtered result records may cross the LLM boundary.

III) **Version consistency:** Schema retrieval, validation, compilation, and execution shall use the same resolved configuration context. Each response includes a schema fingerprint and context label for traceability.

IV) **Bounded resources:** The service rejects empty or excessively long requests and caps returned records. The current default limits are 4,000 question characters, 160 version-selector characters, and 200 returned records.

V) **Statelessness:** The MCP server shall process each request independently. Conversation history and interpretation of follow-up questions remain the responsibility of the calling agent.

VI) **Deployment security:** The CERN server shall bind to a loopback address and be reached from a workstation through an authenticated SSH local port forward rather than a public listener.

The implementation requires Python 3.10 or later, an OpenAI-compatible LLM endpoint configured through environment variables, and a selected OKS data file. Full CERN execution additionally requires an accessible ATLAS TDAQ release and its CVMFS-provided OKS schema and runtime tools.

4.2 System Overview

The implemented system converts a natural-language question into a validated OksQuery expression and executes the expression against an approved OKS configuration. The design follows a schema-RAG approach: instead of placing the entire configuration in an LLM prompt, the server retrieves a small schema slice for the current version and uses it to constrain translation. The following figure shows the implementation and deployment boundaries.

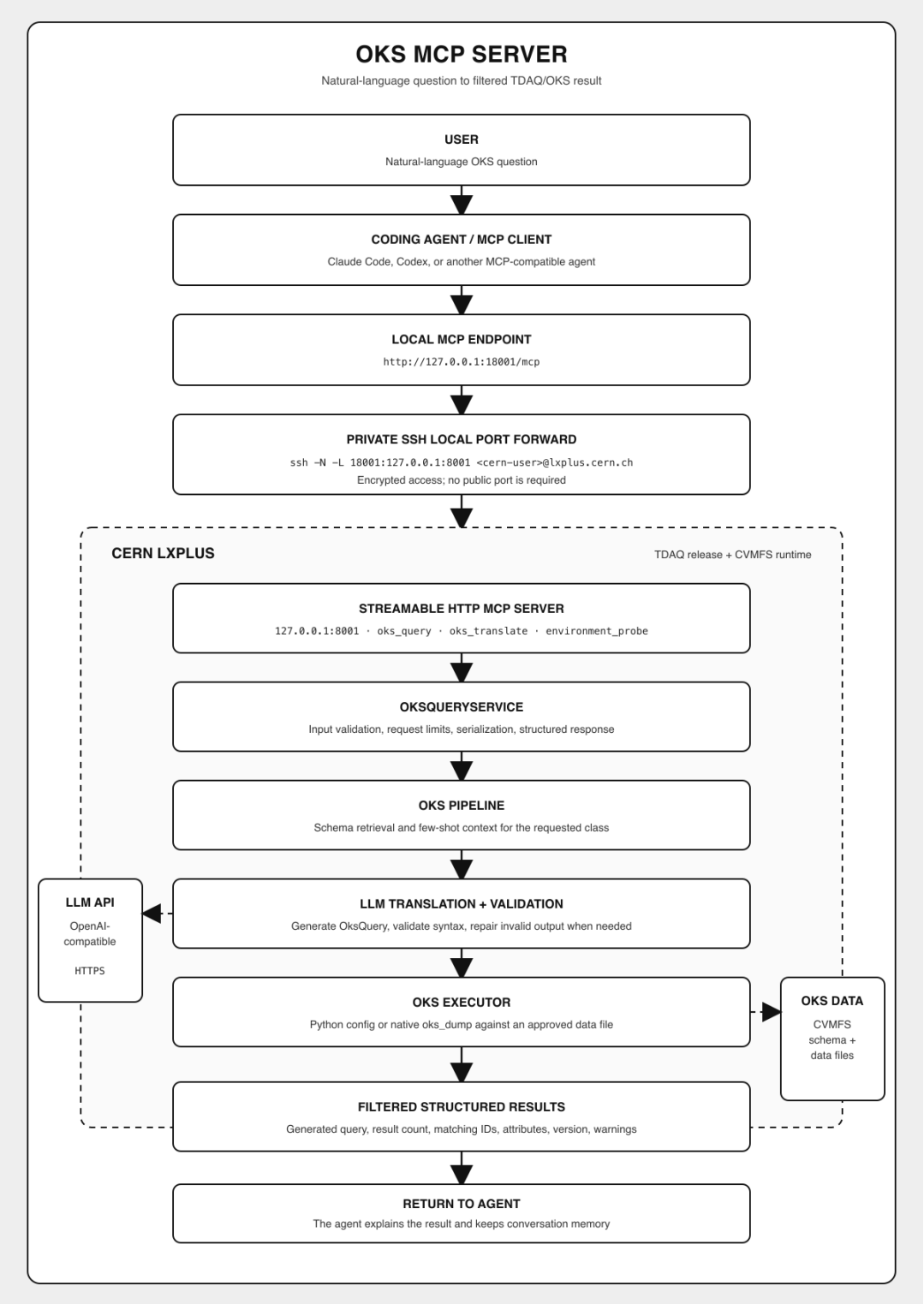

Figure 4.1: Implemented private MCP architecture for natural-language OKS queries.

A user submits a complete question through an MCP-compatible coding agent. For a workstation-based client, the request first reaches a local endpoint and then crosses an SSH local port forward to the loopback-only MCP server on CERN LXPLUS. The server validates the input and invokes the `OksPipeline`. The pipeline resolves the relevant

context, retrieves schema information and few-shot examples, prompts the configured LLM, normalizes and validates the resulting intermediate representation, compiles a valid OksQuery expression, and executes it using the selected OKS configuration. Only filtered structured results return through the MCP server to the agent.

This mapping preserves the project methodology in the deployed implementation. Schema retrieval and few-shot examples reduce unsupported query generation; deterministic validation and repair prevent invalid schema references from reaching execution; and the executor provides the backend filter that keeps the full configuration outside the LLM context window.

4.3 Development Environment

The project was implemented primarily in Python and integrated with the existing ATLAS TDAQ/OKS runtime. Table 4.1 summarizes the principal development environment. Exact processor and memory specifications are not fixed by the repository; they should be recorded with the Chapter 5 experiments if resource-use measurements are reported.

Table 4.1: Development environment and principal tools.

Item	Implementation environment
Programming language	Python 3.10 or later.
Translation and integration	Python package `oksquery_translator`, containing the MCP server, service facade, pipeline, schema retrieval, validation, and execution modules.
External libraries	`openai` for an OpenAI-compatible LLM API, `mcp` for the Model Context Protocol server, `python-dotenv` for environment loading, `rank-bm25` for lexical schema retrieval, and `pytest` for tests.
CERN runtime	CERN LXPLUS/Linux environment with an ATLAS TDAQ release sourced from CVMFS. The current deployment guide uses `tdaq-14-00-00`.
OKS access	Python `config` bindings are preferred for current configurations; the native `oks_dump` command is used as a fallback and for historical configuration paths.
Development and deployment tools	Git, Python virtual environment, SSH with CERN authentication, and `tmux` for a detached development server process.
Configuration	Environment variables select the LLM API endpoint/model, OKS data file, schema directory, MCP transport, host, and port. API credentials are stored outside version control.

The system can use bundled schema material during local development when CVMFS is unavailable. However, experiments that claim native TDAQ execution must be performed in a compatible TDAQ environment with the intended data file and release selected.

4.4 System Implementation

4.4.1 MCP and Service Boundary

The entry point, `mcp_server.py`, uses the FastMCP server implementation. It supports standard input/output transport for a local agent and Streamable HTTP for the CERN-hosted path. The server delegates each tool call to `OksQueryService` in `service.py`. This service is stateless and owns the server-side data-file and schema selection; it vali-

dates the question and optional version selector before a request reaches the translation pipeline. It also serializes pipeline access because version selection can temporarily affect TDAQ environment variables.

The service normalizes results into a common response structure. In addition to query results, this structure includes the target class, generated query, number of translation attempts, selected version, schema fingerprint, context label, and warnings. A successful query returning no objects is explicitly reported as a valid empty result rather than a service failure. Result records are capped to protect the response boundary.

4.4.2 Translation, Validation, and Execution Flow

For every accepted request, the `OksPipeline` integrates the following stages:

1. **Intent and version handling:** The pipeline classifies the question, extracts any run or partition information, and resolves the requested configuration version when available. Questions outside the supported OKS-query scope end early with a structured message.
2. **Context creation and preprocessing:** `OksContextBuilder` constructs a version-bound context. `QueryPreprocessor` analyzes the natural-language input before retrieval.
3. **Schema retrieval:** `SchemaSearchIndex`, `OksSchemaProvider`, and `SchemaRetriever` select the relevant classes and schema elements. The index is partitioned by schema fingerprint so that context from one schema version is not reused for another.
4. **Prompt construction and translation:** `PromptBuilder` combines the retrieved schema slice, few-shot examples, and translation rules. `Translator` asks the configured LLM to produce a structured query representation.
5. **Deterministic safety checks:** The representation is normalized by `normalize_ir`, checked by `ASTValidator` against the exact selected schema, and compiled by `OksCompiler` into an OksQuery S-expression. If validation fails, the diagnostic is passed to the bounded repair loop before compilation is attempted again.
6. **Read-only execution:** `Executor` runs the compiled query. For current configura-

tions, it first attempts the Python `config.Configuration` interface; if required, it falls back to `oks_dump`. The executor returns only matching objects and selected attributes, not the complete configuration.

7. **Response assembly:** The service returns a structured MCP response. The calling agent may explain the result to the user, but the server itself does not store conversation state.

The separation between LLM-assisted translation and deterministic processing is deliberate. The LLM proposes a query representation, while normalization, schema validation, compilation, execution, input limits, and result filtering are implemented in deterministic Python and native OKS components. Consequently, a schema-valid query is required before execution, although semantic correctness must still be assessed by result-set evaluation in Chapter 5.

4.5 Interface and Visualization

The implemented user interface is tool-oriented rather than a graphical dashboard. An MCP-compatible client displays the three available operations: `oks_query`, `oks_translate`, and `oks_environment_probe`. The main query tool accepts a complete natural-language question and optionally a version selector. Its response exposes the generated OksQuery, target class, exact result count, filtered object records, version/context metadata, and any warnings. This format makes the translation and execution path inspectable by both users and developers.

The runtime evidence in Figures 4.2–4.4 shows the private connection and client-side interface. Sensitive usernames, hostnames, paths, and the second-factor code were removed from the derivative figures before inclusion.

Figure 4.2: MCP service running on the LXPLUS loopback endpoint at port 8001.

Figure 4.3: Redacted workstation command establishing the SSH local port forward.

Figure 4.4: MCP client showing the connected `oksquery` server and its three tools.

4.6 Deployment and Execution

The documented deployment is a private development arrangement rather than a publicly exposed production service. The MCP process runs on CERN LXPLUS with the selected TDAQ environment sourced and the server bound to `127.0.0.1:8001`. Binding to loopback prevents the MCP server from accepting direct connections from the public network.

The deployment sequence is as follows:

1. On LXPLUS, clone the project, create and activate the project virtual environment, install the package requirements, source the selected TDAQ setup script, and configure the LLM credentials through a local `.env` file.
2. Select an approved `OKS_DATA_FILE`, configure `MCP_TRANSPORT` as `streamable-http`, set `MCP_HOST` to `127.0.0.1`, and set `MCP_PORT` to `8001`.
3. Start the server using `bash deploy/start_mcp_tmux.sh`. The `tmux` session keeps the process available during development and allows its logs to be inspected.
4. On the workstation, create an authenticated SSH local port forward from local port 18001 to the remote loopback endpoint: `ssh -N -L 18001:127.0.0.1:8001 <cern_user>@<lxplus_host>`.
5. Configure the MCP client to use the forwarded Streamable HTTP endpoint `http://127.0.0.1:18001/mcp`, then verify the three MCP tools and submit a complete test question.

The SSH tunnel uses the user's existing CERN SSH authentication and keeps the service private to the workstation that opened the forward. If the MCP client runs directly on

LXPLUS, the server can instead be configured through standard input/output and the SSH tunnel is unnecessary. A temporary public tunnel is not appropriate for the current implementation because the MCP HTTP boundary does not yet provide authentication or authorization.

The present `tmux`-based process is suitable for development and demonstration but not a production supervisor. Shared or public deployment requires an approved service owner, restart policy, authentication and authorization, TLS/reverse-proxy or managed-tunnel policy, rate limiting, and privacy-conscious logging.

4.7 Challenges and Solutions

4.7.1 Large and Evolving Configuration Context

The OKS schema and configuration contain substantially more information than can be sent safely and efficiently in a single LLM prompt. Supplying the full configuration would increase context cost and make relevant information difficult to locate. The implemented solution is schema-RAG: the pipeline retrieves only the schema slice relevant to the question and associates that slice with a schema fingerprint. This preserves the version boundary and reduces the context presented to the LLM.

4.7.2 Proprietary Query Syntax and LLM Hallucination

OksQuery uses a strict S-expression syntax and schema-dependent class, attribute, relationship, and comparator names. An LLM can produce plausible but invalid expressions. The system addresses this with few-shot examples, a structured intermediate representation, deterministic normalization, schema-bound validation, compilation, and a bounded repair loop driven by validation diagnostics. This prevents a syntactically or schema-invalid query from reaching the executor. It does not by itself prove that a valid query captures the intended meaning; result-set comparison is therefore required during evaluation.

4.7.3 Safe Access to Configuration Data

The underlying TDAQ environment includes powerful APIs, but this project must not expose modification operations or arbitrary server access through an LLM tool. The MCP service restricts requests to complete natural-language questions and approved version selectors, keeps data-file selection server-side, limits request and response sizes,

and exposes only read-only tools. The executor returns matching objects rather than the complete configuration.

4.7.4 Version and Runtime Compatibility

A query is meaningful only relative to the schema and configuration version from which it was derived. Mixing a schema from one release with data from another can yield failures or misleading answers. The implementation builds a version-bound context per query, fingerprints the schema used for retrieval and validation, and records version metadata in the response. Historical access remains dependent on availability of the required TDAQ release, repository configuration, data-file path, and permitted version selector.

4.7.5 Private Connectivity Between Workstation and CERN

An external coding agent cannot directly reach a loopback-only service on LXPLUS. The system uses SSH local port forwarding to expose the remote loopback endpoint only as a local workstation endpoint. This retains CERN SSH authentication and avoids a public listener. The trade-off is that each user must keep an individual SSH tunnel active; a future shared deployment requires a separately approved authenticated service boundary.

4.7.6 Distinguishing Empty Results from Failures

A query can be valid and execute correctly while matching no objects in the selected configuration. Treating this condition as a failure would cause unnecessary retries and make debugging difficult. The service explicitly marks a zero-count successful result as a valid empty result and returns an explanatory warning, allowing the calling agent to decide whether the user wishes to broaden the request.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Results

5.1.1 Verification of the Implementation

The implementation was verified at several levels. The repository test record after the report branch integration contains 199 passing tests and 9 skipped tests. The Python package also passed compilation and whitespace checks. A real MCP client initialized the server, discovered the three declared tools, and completed tool calls. In the CERN runtime, the selected TDAQ release was `tdaq-14-00-00`; the Streamable HTTP server listened on `127.0.0.1:8001`, and the workstation reached it through the forwarded endpoint `http://127.0.0.1:18001/mcp`.

The end-to-end path was also exercised against an OKS data fixture. The verified runtime notes record a successful query returning two `Computer` objects through the native execution path. This confirms that the MCP transport, translation pipeline, validation boundary, and OKS executor can be connected in the intended environment.

5.1.2 Observed Query Demonstrations

Figure 5.1 shows a valid query for an object that is not present in the selected data version. The generated expression is `(all (object-id "rc_trigger_1" =))`; the service reports result count zero and explicitly labels the outcome as a valid empty result. This is different from a transport or execution failure and prevents the agent from performing speculative retries.

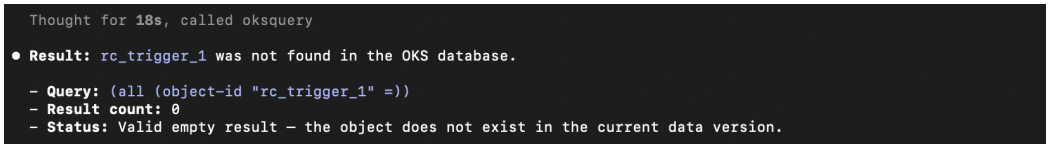

Figure 5.1: Valid empty result returned for an object absent from the selected configuration.

Figure 5.2 shows a natural-language request for `BaseApplication` objects with `InitTimeout > 30`. The client displays the generated query `(all ("InitTimeout" "30" >))`, an exact result count of 16, and the returned object identifiers. The screenshot also reports that the returned objects have `InitTimeout = 60`. This is qualita-

tive evidence that a schema-grounded query can be translated, executed, and returned through the MCP client with inspectable intermediate output.

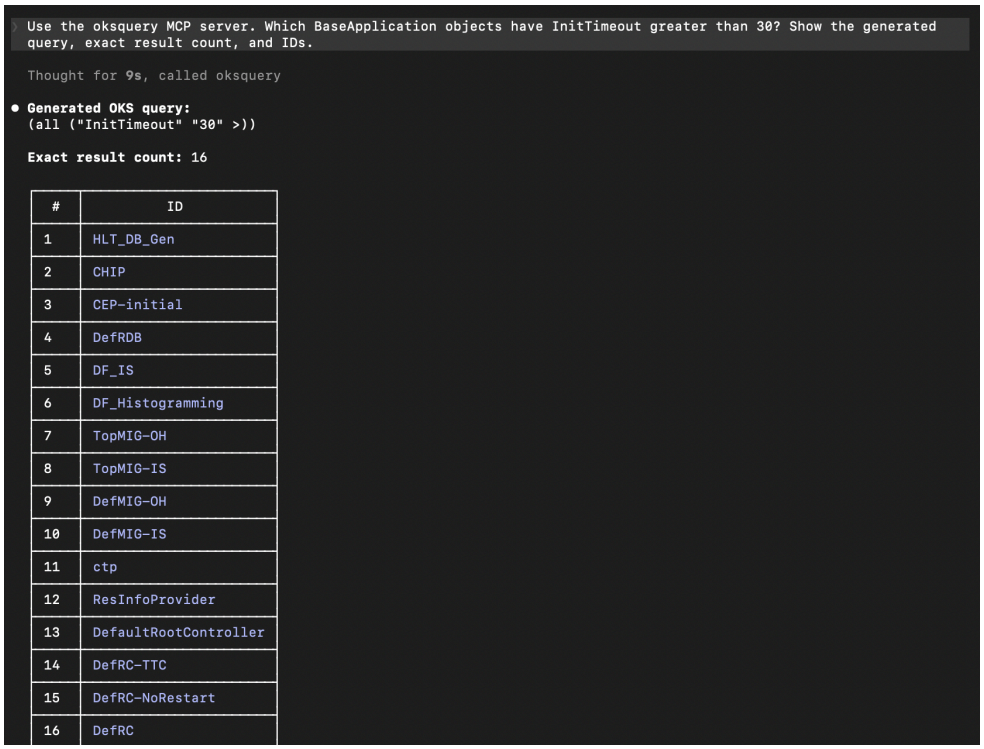

Figure 5.2: Successful BaseApplication query showing generated OksQuery, count, and identifiers.

5.2 Discussion

The demonstrations establish the core integration objective: an external agent can submit natural language, the server can expose the generated OksQuery and validation outcome, and only filtered results return from the CERN-hosted configuration. The explicit empty-result status is particularly useful because it preserves the distinction between an unsuccessful request and a successful query with no matching object.

These results should not be interpreted as a measured translation-accuracy or latency benchmark. The screenshots cover representative cases rather than a statistically sampled evaluation set, and the report does not claim that every natural-language formulation is translated correctly. Semantic correctness still requires a labelled dataset, expected result sets, and repeated measurements across query difficulty and configuration versions.

The main contribution demonstrated here is integration feasibility. The transport layer, SSH forwarding arrangement, MCP tool contract, translation pipeline, schema valida-

tion boundary, and OKS execution path operate together in the intended environment. Because the generated OksQuery, selected class, result count, warnings, and configuration context are returned explicitly, the calling agent can inspect the result and distinguish an input, validation, transport, or execution problem from a successful query.

The valid empty-result demonstration also has operational value. A zero result count does not necessarily indicate a failed request; it may reflect an absent object, a restrictive predicate, or a different configuration version. Reporting this case explicitly prevents unnecessary retries, although deciding how to correct the question remains the responsibility of the user or calling agent. Similarly, the separation between LLM-assisted translation and deterministic retrieval, validation, compilation, and execution reduces the chance that an invalid schema reference reaches OKS. The version-bound context and schema fingerprint improve traceability, but a schema-valid query is not necessarily semantically correct.

The private loopback and SSH-forwarded deployment keeps the complete OKS configuration on the CERN side and exposes only filtered responses. It is appropriate for the demonstrated controlled environment, but still depends on the selected TDAQ release, OKS data file, CVMFS, an active SSH tunnel, and an LLM endpoint. It should therefore not be considered a production deployment; production use would require approved supervision, stronger service authentication and authorization, operational logging, and recovery controls.

These results are not a translation-accuracy or latency benchmark. A stronger evaluation should use a labelled set of questions with canonical queries and expected result sets across query difficulties and configuration versions. It should report schema-validity and execution rates, result-set equivalence, repair frequency, and end-to-end latency, preferably against a direct-generation baseline. Until such testing is completed, the demonstrations support the feasibility of an inspectable, version-aware, read-only integration path rather than broad claims about general translation accuracy or production robustness.

CHAPTER 6

CONCLUSIONS AND RECOMMENDATIONS

6.1 Conclusions

This project implemented a read-only Natural Language to OksQuery translation module for the ATLAS OKS configuration environment. The final system combines schema-RAG retrieval, few-shot prompting, structured query generation, deterministic validation, bounded repair, and controlled execution behind an MCP service boundary. The service is stateless, limits request and result size, and keeps the full configuration on the CERN side of the deployment.

The verified demonstrations show that the intended request path works through the private deployment: an MCP client discovers the service, submits a natural language question through an SSH-forwarded endpoint, receives an inspectable OksQuery, and obtains filtered OKS results. The system also distinguishes valid empty results from failures. These outcomes support the feasibility of the architecture, while the absence of a large labelled benchmark limits the strength of any claim about general translation accuracy.

6.2 Recommendations/Future Enhancements

Future work should proceed in the following order:

(i) Build and label a stratified evaluation dataset covering simple attribute predicates, compound predicates, object identifiers, relationship traversal, inherited attributes, and unsupported requests.

(ii) Measure exact execution accuracy, result-set equivalence, first-pass validation rate, repair success rate, rejection precision, and end-to-end latency across multiple configuration versions.

(iii) Add ablation experiments for direct prompting, few-shot prompting, schema retrieval, structured IR, and validate/repair so that each component's contribution is measured rather than inferred.

(iv) Add stronger service authentication and authorization before any shared or public deployment. Replace the development `tmux` process with an approved supervisor and define logging, rate-limit, and data-retention policy.

(v) Improve result extraction and schema retrieval for relationship-heavy queries, and add regression tests whenever the TDAQ release or OKS schema changes.

REFERENCES

[1] R. Jones, L. Mapelli, Y. Ryabov, and I. Soloviev, "The OKS persistent in-memory object manager," *IEEE Transactions on Nuclear Science*, vol. 45, no. 4, pp. 1958–1964, Aug. 1998. [Online]. Available: `https://cds.cern.ch/record/370902/files/cer-000296198.pdf`

[2] J. Almeida, M. Dobson, A. Kazarov, G. Lehmann Miotto, J. E. Sloper, I. Soloviev, and R. C. Torres, "The ATLAS DAQ system online configurations database service challenge," *Journal of Physics: Conference Series*, vol. 119, no. 2, Art. no. 022004, 2008, doi: 10.1088/1742-6596/119/2/022004.

[3] I. Soloviev, "The version control service for the ATLAS data acquisition configuration files," *Journal of Physics: Conference Series*, vol. 396, Art. no. 012047, 2012, doi: 10.1088/1742-6596/396/1/012047.

[4] I. Soloviev, "The git based ATLAS data acquisition configuration service in LHC Run 3," *EPJ Web of Conferences*, vol. 337, Art. no. 01038, 2025, doi: 10.1051/epjconf/202533701038.

[5] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: `https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html`

[6] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901. [Online]. Available: `https://arxiv.org/abs/2005.14165`

[7] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-SQL," *The VLDB Journal*, vol. 32, pp. 905–936, 2023, doi: 10.1007/s00778-022-00776-8.

[8] X. Zhu, Y. Xie, Y. Liu, Y. Li, and W. Hu, "Knowledge Graph-Guided Retrieval Augmented Generation," arXiv:2502.06864, NAACL, 2025.

APPENDICES

Appendix A: Demonstrated Runtime Configuration

The runtime demonstration used a CERN LXPLUS host with TDAQ release `tdaq-14-00-00`. The MCP server was configured for Streamable HTTP on `127.0.0.1:8001`; the workstation forward used local port 18001 and the client endpoint `http://127.0.0.1:18001/mcp`. The service exposed `oks_query`, `oks_translate`, and `oks_environment_probe`. Credentials and host-specific identifiers are intentionally omitted from this report.

Appendix B: Evidence and Reproducibility Notes

The report figures are derived from runtime screenshots and were cropped or redacted before inclusion. The screenshots demonstrate service startup, private transport, tool discovery, a valid empty result, and a 16-object query. They do not by themselves establish general model accuracy, production availability, or performance across all TDAQ partitions. Reproduction requires access to a compatible CERN environment, the selected TDAQ release, an approved OKS data file, and an authorized LLM endpoint.